

## Anexo 08 — Evaluacion de cabeza con objetivos extremos

Archivo: Scripts/eval\_v14\_head.py

### Para que sirvio:

Evaluacion especifica con objetivos en azimuth extremo (hasta  $\pm 150$  grados) para verificar que la cabeza usa el giro correctamente. Genera video con overlay de los angulos de cada articulacion.

```
1 | """Eval custom para v14_head: testea targets en azimuth EXTREMO (incluye atras).
2 |
3 | Por que: eval_ppo.py usa el cono default del env (TARGET_CONE_AZIMUTH=75°),
4 | asi que NUNCA muestrea targets atras del robot. v14_head fue entrenada con cono
5 | extendido a  $\pm 150^\circ$  – necesitamos targets en ese rango para validar el cambio.
6 |
7 | Genera 8 episodios con azimuth explicito (sweep -150°→+150°), renderiza video
8 | con overlay mostrando target azimuth y angulo theta del head.
9 |
10 | Uso:
11 |     python Scripts/eval_v14_head.py
12 |         [--model runs/ppo_dum_v14_head/final.zip]
13 |         [--video runs/ppo_dum_v14_head/eval_behind.mp4]
14 | """
15 | import argparse
16 | import sys
17 | from pathlib import Path
18 |
19 | SCRIPTS_DIR = Path(__file__).resolve().parent
20 | sys.path.insert(0, str(SCRIPTS_DIR))
21 |
22 | import numpy as np
23 | import mujoco
24 | from PIL import Image, ImageDraw, ImageFont
25 | from stable_baselines3 import PPO
26 |
27 | from rl_env import DUMHeadTrackingEnv, TARGET_DISTANCE_MIN, TARGET_DISTANCE_MAX
28 |
29 |
30 | def main():
31 |     p = argparse.ArgumentParser()
32 |     p.add_argument("--model", type=str, default="runs/ppo_dum_v14_head/final.zip")
33 |     p.add_argument("--video", type=str, default="runs/ppo_dum_v14_head/eval_v14_behind.mp4")
34 |     p.add_argument("--steps-per-target", type=int, default=400,
35 |                    help="Steps por target (8s a 50Hz). 400 = suficiente para que HeadRot llegue al limite (6.6s).")
36 |     args = p.parse_args()
37 |
38 |     # Lista explicita de target azimuth a probar (en grados)
39 |     target_azs = [0, 45, 90, 120, 150, -90, -120, -150]
40 |     target_el = 10 # elevation moderada para todos
41 |
42 |     print(f"[setup] cargando {args.model}")
43 |     model = PPO.load(args.model)
44 |
45 |     env = DUMHeadTrackingEnv()
46 |     obs, _ = env.reset(seed=42)
47 |     print(f"[setup] env OK. obs.shape={obs.shape}")
48 |
49 |     renderer = mujoco.Renderer(env.model, height=480, width=480)
50 |     try:
51 |         font = ImageFont.truetype("arial.ttf", 13)
52 |         font_small = ImageFont.truetype("arial.ttf", 11)
53 |     except Exception:
54 |         font = ImageFont.load_default()
55 |         font_small = font
56 |
57 |     frames = []
58 |     summary = []
59 |
60 |     def target_world_for(az_deg, el_deg, dist=0.6):
61 |         az = np.deg2rad(az_deg)
62 |         el = np.deg2rad(el_deg)
63 |         # Replica de _sample_point_in_cone con valores fijos
64 |         target_dir = env._rot_z(az) @ env._rot_x(el) @ env._forward_world_init
65 |         target_dir /= np.linalg.norm(target_dir) + 1e-9
66 |         return env._head_pos_init + dist * target_dir
67 |
68 |     for az_deg in target_azs:
69 |         # Reset env y forzar target estatico
70 |         obs, _ = env.reset(seed=int(abs(az_deg) * 10) + 1)
71 |         target_world = target_world_for(az_deg, target_el)
72 |         env._target_mode = "static"
73 |         env._target_a = target_world
74 |         env._target_world = target_world.copy()
```

```

75 |     env.data.mocap_pos[env._target_mocap_idx] = target_world
76 |     mujoco.mj_forward(env.model, env.data)
77 |
78 |     # Trackeo
79 |     theta_history = []
80 |     head_rot_history = []
81 |     head_base_history = []
82 |     for step in range(args.steps_per_target):
83 |         obs = env._get_obs()
84 |         action, _ = model.predict(obs, deterministic=True)
85 |         obs, r, term, trunc, info = env.step(action)
86 |
87 |         theta = env._angle_to_target()
88 |         theta_history.append(theta)
89 |         head_rot_history.append(float(env.data.qpos[env._head_qpos_adr[2]])) # HeadRotation_joint
90 |         head_base_history.append(float(env.data.qpos[env._head_qpos_adr[1]])) # HeadBase_joint
91 |
92 |     # Render
93 |     renderer.update_scene(env.data, camera=-1)
94 |     img = Image.fromarray(renderer.render())
95 |     d = ImageDraw.Draw(img)
96 |     head_rot_deg = np.rad2deg(env.data.qpos[env._head_qpos_adr[2]])
97 |     head_base_deg = np.rad2deg(env.data.qpos[env._head_qpos_adr[1]])
98 |     lines = [
99 |         "v14_head - eval con targets extremos",
100 |         f"target az={az_deg:+4d}° el={target_el}°",
101 |         f"step {step:3d}",
102 |         f"theta_to_target: {np.rad2deg(theta):5.1f}°",
103 |         f"HeadRot (yaw): {head_rot_deg:+6.1f}°",
104 |         f"HeadBase(pitch): {head_base_deg:+6.1f}°",
105 |     ]
106 |     for i, ln in enumerate(lines):
107 |         color = (100, 255, 100) if (np.rad2deg(theta) < 10) else (255, 240, 100)
108 |         d.text((6, 6 + i * 14), ln, fill=color, font=font_small)
109 |     frames.append(np.array(img))
110 |
111 |     # Stats finales (ultimo segundo del episodio = settling)
112 |     last_50 = theta_history[-50:]
113 |     last_50_rot = head_rot_history[-50:]
114 |     last_50_base = head_base_history[-50:]
115 |     theta_final = float(np.mean(last_50))
116 |     rot_final = float(np.mean(last_50_rot))
117 |     base_final = float(np.mean(last_50_base))
118 |     summary.append({
119 |         "az": az_deg,
120 |         "theta_deg_final": np.rad2deg(theta_final),
121 |         "head_rot_deg_final": np.rad2deg(rot_final),
122 |         "head_base_deg_final": np.rad2deg(base_final),
123 |     })
124 |     print(f" az={az_deg:+4d}°: theta_final={np.rad2deg(theta_final):5.1f}° "
125 |           f"HeadRot={np.rad2deg(rot_final):+6.1f}° HeadBase={np.rad2deg(base_final):+6.1f}°")
126 |
127 |     # Video
128 |     import imageio
129 |     out_path = Path(args.video)
130 |     out_path.parent.mkdir(parents=True, exist_ok=True)
131 |     imageio.mimsave(str(out_path), frames, fps=50, quality=8)
132 |     print(f"\nVideo: {out_path} ({len(frames)} frames @ 50fps)")
133 |
134 |     # Diagnostico
135 |     print("\n=== DIAGNOSTICO ===")
136 |     for s in summary:
137 |         # Para tracking correcto: HeadRot deberia ser cercano al azimuth del target
138 |         # HeadBase deberia ser cercano a el target_el (no perpendicular al piso)
139 |         rot_ok = abs(s["head_rot_deg_final"] - s["az"]) < 30
140 |         head_stable = abs(s["head_base_deg_final"]) < 40 # no tilteado al piso
141 |         flag = "OK" if (s["theta_deg_final"] < 15 and rot_ok and head_stable) else "FAIL"
142 |         print(f" az={s['az']:+4d}° theta={s['theta_deg_final']:5.1f}° "
143 |               f"rot={s['head_rot_deg_final']:+6.1f}° base={s['head_base_deg_final']:+6.1f}° [{flag}]")
144 |
145 |
146 | if __name__ == "__main__":
147 |     main()
148 |

```